Nathan Krieger
S E 417
Extra Credit Assignment

(a) Select two of these bugs and provide details of the bug (the problem/references to bug number) and a description summarizing both the problem and the fixes. At least One of these should be a critical (or blocker) bug. You will need to play with the search facility to identify these. Find the patches for each bug and provide a screenshot of the diff for the changes as well. (55 points)

Bug 1:

Bug 113994 - [13 Regression] Probable C++ code generation bug with -O2 on s390x platform

This bug is a compiler miscompilation issue in GCC affecting C++ programs when the optimization level -O2 is enabled.

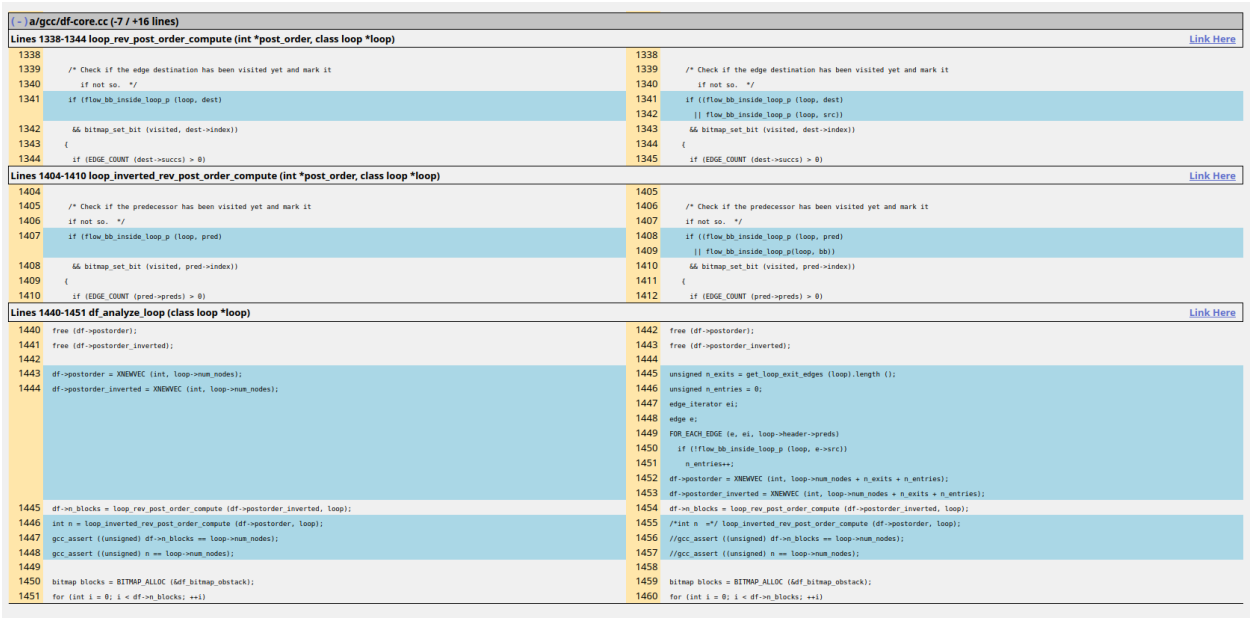
A valid C++ program throws a std::out_of_range exception during execution.
The exception occurs inside std::string::substr(), even though the parameters are valid.
The same program behaves correctly under:
-O0 and -O1
Other compilers
Other CPU architectures

This shows that the compiler is generating incorrect machine code under optimization, rather than the program itself being incorrect.

From the bug report:

The failure only occurs on the s390x architecture
The issue disappears if minor changes are made, which suggests an optimization bug

Patch:



(-) a/gcc/df-core.cc (-7 / +16 lines)

Lines 1338-1344 loop_rev_post_order_compute (int *post_order, class loop *loop) Link Here

```
1338                                                    1338
1339     /* Check if the edge destination has been visited yet and mark it    1339     /* Check if the edge destination has been visited yet and mark it
1340        if not so.  */                                1340        if not so.  */
1341     if (flow_bb_inside_loop_p (loop, dest)           1341     if ((flow_bb_inside_loop_p (loop, dest)
                                                         1342       || flow_bb_inside_loop_p (loop, src))
1342       && bitmap_set_bit (visited, dest->index))      1343       && bitmap_set_bit (visited, dest->index))
1343     {                                               1344     {
1344       if (EDGE_COUNT (dest->succs) > 0)             1345       if (EDGE_COUNT (dest->succs) > 0)
```

Lines 1404-1410 loop_inverted_rev_post_order_compute (int *post_order, class loop *loop) Link Here

```
1404                                                    1405
1405     /* Check if the predecessor has been visited yet and mark it        1406     /* Check if the predecessor has been visited yet and mark it
1406     if not so.  */                                  1407     if not so.  */
1407     if (flow_bb_inside_loop_p (loop, pred)          1408     if ((flow_bb_inside_loop_p (loop, pred)
                                                         1409       || flow_bb_inside_loop_p(loop, bb))
1408       && bitmap_set_bit (visited, pred->index))     1410       && bitmap_set_bit (visited, pred->index))
1409     {                                               1411     {
1410       if (EDGE_COUNT (pred->preds) > 0)             1412       if (EDGE_COUNT (pred->preds) > 0)
```

Lines 1440-1451 df_analyze_loop (class loop *loop) Link Here

```
1440   free (df->postorder);                            1442   free (df->postorder);
1441   free (df->postorder_inverted);                   1443   free (df->postorder_inverted);
1442                                                    1444
1443   df->postorder = XNEWVEC (int, loop->num_nodes);  1445   unsigned n_exits = get_loop_exit_edges (loop).length ();
1444   df->postorder_inverted = XNEWVEC (int, loop->num_nodes);  1446   unsigned n_entries = 0;
                                                         1447   edge_iterator ei;
                                                         1448   edge e;
                                                         1449   FOR_EACH_EDGE (e, ei, loop->header->preds)
                                                         1450     if (!flow_bb_inside_loop_p (loop, e->src))
                                                         1451       n_entries++;
                                                         1452   df->postorder = XNEWVEC (int, loop->num_nodes + n_exits + n_entries);
                                                         1453   df->postorder_inverted = XNEWVEC (int, loop->num_nodes + n_exits + n_entries);
1445   df->n_blocks = loop_rev_post_order_compute (df->postorder_inverted, loop);  1454   df->n_blocks = loop_rev_post_order_compute (df->postorder_inverted, loop);
1446   int n = loop_inverted_rev_post_order_compute (df->postorder, loop);  1455   /*int n  =*/ loop_inverted_rev_post_order_compute (df->postorder, loop);
1447   gcc_assert ((unsigned) df->n_blocks == loop->num_nodes);  1456   //gcc_assert ((unsigned) df->n_blocks == loop->num_nodes);
1448   gcc_assert ((unsigned) n == loop->num_nodes);    1457   //gcc_assert ((unsigned) n == loop->num_nodes);
1449                                                    1458
1450   bitmap blocks = BITMAP_ALLOC (&df_bitmap_obstack);  1459   bitmap blocks = BITMAP_ALLOC (&df_bitmap_obstack);
1451   for (int i = 0; i < df->n_blocks; ++i)           1460   for (int i = 0; i < df->n_blocks; ++i)
```

The issue was that GCC was analyzing which basic blocks belong in a loop during dataflow analysis incorrectly. It was only looking at nodes inside the loop and ignored boundary conditions like entry and exit edges (to and from the loop).

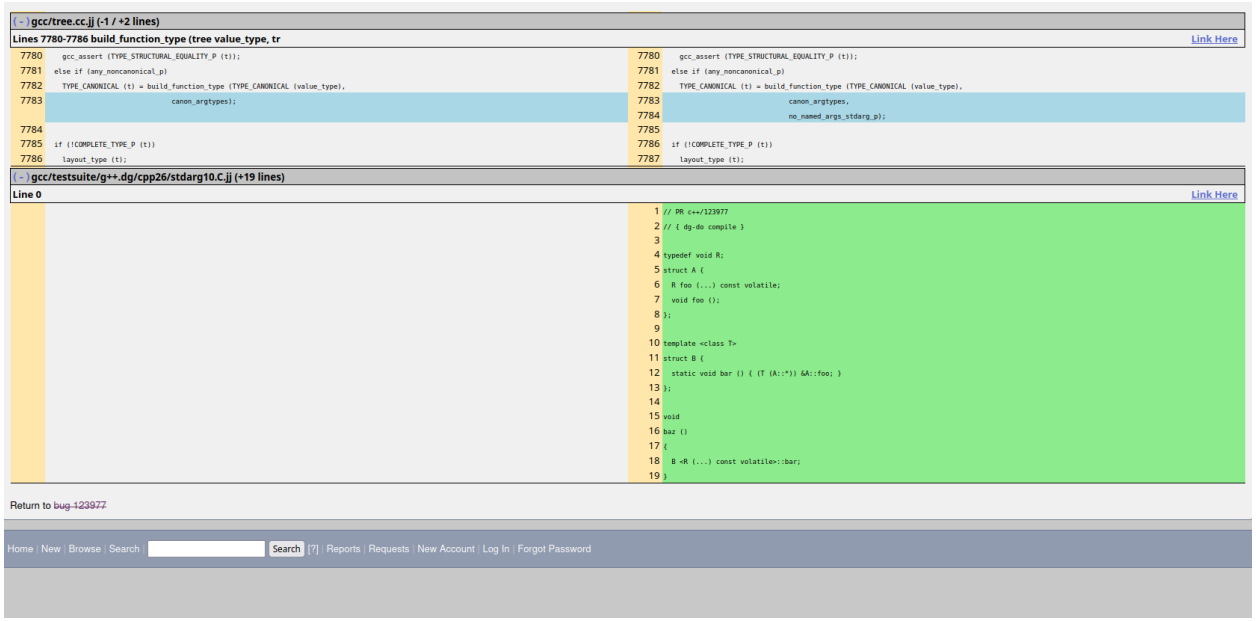
Fix:

This bug was fixed by including more nodes in the loop. Before the edges were only processed when their destination was inside the loop, now it processes edges when their source is inside of it.

Bug 2 (P1 - blocker):

Bug 123977 - [16 Regression] ICE on member function pointer in C++26 since r16-4338

This bug is an Internal Compiler Error triggered when compiling valid C++26 code.
GCC incorrectly compares or constructs function types involving void*&(...) const volatile and overloaded member functions which leads to a mismatch that causes the compiler to crash



The fix was simple. It just added no_named_args_stdarg_p to the parameter. This ensured correct propagation of no_named_args_stdarg_p because of C++26.

We can also see in the fix that a test was added to ensure this would get caught in the future.

(b) Use 2 concepts/techniques from this class (your choice) and use the example bugs and to explain how these concepts could have helped you to find the bugs you are describing. This should be very specific with relation to the examples from part (a). It is not enough to say “use technique X to find a bug or measure code” or “use regression testing to retest”. We are looking for details about how the technique could have been applied and why it might help the bugs from part a. (45 points)

Bug 1:

Technique 1: Boundary Tests

We learned about boundry testing in a more simpler manner. For example we learned that when testing the elements of a list, the 0th, 1st, n-1, and nth positions should be tested. I believe the same idea can be applied here. On a CFG, the boundary conditions would be where a loop is entered and exited. This bug was caused by not properly handling a testing loop boundaries so more robust testing would have found this bug quickly. Some tests we could write would be a loop that runs either 0 or 1 times and code where a variable is defined right before or right after the loop.

Technique 2: Data flow tests

Focusing on definition use cases that we leaned in this course, we could write tests that ensure values are correct along their paths. This is relevant to this bug since GCC incorrectly tracked how values moved through the loop. We could track variables defined before, during and after loops and compare their values with -O0 and -O2 to see the difference.

Bug 2:

Technique 1: Regression Testing

We know that this bug was caused by new behaviors of existing code because of C++26. A strong test on the existing program would have caught this. We can also see that after hte fix was made, a test was added that may be trivial now, but its role could be important if another breaking change is made in the future.

Technique 2: Metamorphic Testing

During SE 417 we learned that metamorphic testing checks the relationship of oracles rather than the oracles themselves. This concept could be applied here by testing two semantically identical types and seeing if GCC treats them the same across several scenarios.
The simple test could be defining
T1 = void *&(...) const volatile
T2 = void *&(...) const volatile

And then after template substitution ensuring that
T1 == T2